



SKYLOCK CYBER DEFENSE

Application Security Practice | Unbreakable Cyber Security



Web Application Penetration Test Report

Cross-Site Scripting (XSS) -- Security Level Assessment

Target System	DVWA -- Damn Vulnerable Web Application
Security Levels	Medium and Impossible
Prepared By	Junior Penetration Tester -- SkyLock Cyber Defence
Date	1 May 2026
Version	1.0 -- Final
Reference	SKL-PT-2026-001
Classification	CONFIDENTIAL -- TLP: RED



Table of Contents

1. Executive Summary	4
2. Introduction and Objective.....	5
Objectives	5
3. Scope and Methodology.....	5
3.1 Scope	5
3.2 Testing Tools	6
3.3 Methodology	6
Phase 1 -- Environment Configuration	6
Phase 2 -- Reconnaissance	7
Phase 3 -- Active Exploitation (WSTG-INPV-01, WSTG-INPV-02)	7
Phase 4 -- Source Code Review	8
3.4 Rules of Engagement	8
4. Findings and Evidence	8
F-01: Reflected XSS -- Mixed-Case Bypass.....	8
Description	9
Evidence -- Step 1: Standard Payload Blocked	9
Evidence -- Step 2: Mixed-Case Bypass Executed	10
Impact.....	11
Remediation	11
Source Code Analysis	11
F-02: Stored XSS -- Name Field Not Encoded	12
Description	13
Evidence.....	13
Impact.....	13
Remediation	14
Source Code Analysis	14
F-03: No CSRF Protection at Medium Level.....	15
Description	16
Evidence.....	16
Remediation	17
F-04: Raw Database Queries -- SQL Injection Risk	17
Remediation	17
F-05: Browser-Only Length Limits	17



Remediation	18
4.6 Impossible Level -- All Vulnerabilities Closed	18
5. Risk Summary and Findings Table	19
Key Observations	20
6. Mitigation Analysis	21
6.1 Why Medium Security Failed -- Input Filtering vs Output Encoding	21
6.2 Why Impossible Security Succeeded -- Output Encoding.....	21
6.3 Medium vs Impossible -- Security Control Comparison	22
6.4 Content Security Policy as Complementary Defence	22
6.5 Secure Coding Principle	23
7. Recommendations	23
General Guidance	24
8. Reflection.....	24
8.1 How This Assessment Shaped Testing Strategy	24
8.2 Questions This Assessment Raised	25
8.3 Security Patterns to Prioritise in Future Assessments	25
8.4 Personal Growth	25
9. Conclusion	26
10. References.....	27
11. Glossary.....	27



1. Executive Summary

OVERALL RISK RATING: HIGH | 1 Critical · 2 High · 1 Medium · 1 Low

SkyLock Cyber Defence carried out a security test on a legacy HR web application. The test looked at two specific areas: a comment form and a name search field. The goal was simple -- find out whether an outsider could plant malicious code in either field that would then run silently in other users' browsers.

Two protection settings were tested. At the lower setting, both areas were found to be open to attack. The search field was successfully exploited during testing using a basic technique that required no special tools or knowledge. At the higher setting, every attack attempt was blocked.

The confirmed attack on the search field carries real business risk. Anyone who clicks a malicious link sent by an attacker would have their login stolen. On an HR platform, that means the attacker gains access to payroll records, staff personal data, and admin functions -- without ever needing a password.

The fixes are straightforward. The two most urgent issues each require changing a single line of code. All five issues found, their severity, and the recommended actions are listed in the table below.

Ref	Finding	Severity	CVSS	OWASP	Status/Action
F-01	Reflected XSS -- Mixed-Case Bypass	Critical	8.8	A03:2021	Open -- Immediate
F-02	Stored XSS -- Name Field Not Encoded	High	8.0	A03:2021	Open -- Immediate
F-03	No CSRF Protection (Medium Level)	High	7.1	A05:2021	Open -- Short Term
F-04	Raw Database Queries -- SQLi Risk	Medium	5.4	A03:2021	Open -- Planned
F-05	Browser-Only Length Limits	Low	3.1	A04:2021	Open -- Planned

Table 1: Executive Summary of Findings with CVSS Scores and OWASP Classification

Total: 5 Findings | Critical: 1 | High: 2 | Medium: 1 | Low: 1



2. Introduction and Objective

SkyLock Cyber Defence was engaged to conduct a penetration test against a legacy HR management platform ahead of a security compliance audit. All testing took place in a fully authorised, isolated lab environment using DVWA (Damn Vulnerable Web Application). No production systems were accessed at any stage.

The assessment focused on Cross-Site Scripting (XSS), classified under **OWASP A03:2021 Injection as CWE-79**. XSS allows an attacker to inject malicious scripts into web pages viewed by other users. Two variants were assessed: Stored XSS, where the payload persists in the database and fires automatically for every subsequent visitor; and Reflected XSS, where the payload is embedded in a URL and fires when that link is clicked. Stored XSS carries higher inherent risk because no further attacker action is required once the payload is planted.

Beyond testing exploitability at runtime, the server-side PHP source code was reviewed at each security level to explain precisely why each control succeeds or fails. This dual approach -- runtime testing combined with code review -- produces findings that are both evidence-backed and analytically grounded, consistent with the **OWASP Web Security Testing Guide (WSTG v4.2)**.

Objectives

- Determine whether Stored XSS and Reflected XSS remain exploitable at Medium and Impossible security levels.
- Identify the exact server-side sanitisation applied at each level and explain why it succeeds or fails.
- Provide evidence-backed, technically grounded findings with practical code-level remediation guidance.
- Connect observed vulnerability behaviours to secure coding principles and OWASP prevention standards.

3. Scope and Methodology

3.1 Scope



Target Application	DVWA v1.10 (Damn Vulnerable Web Application)
Modules Tested	Stored XSS (/dvwa/vulnerabilities/xss_s/) and Reflected XSS (/dvwa/vulnerabilities/xss_r/)
Web Server	Apache 2.4.63 (Debian) on port 80
Database	MariaDB 11.4.5
Security Levels	Medium and Impossible (Low retained as baseline reference)
Test Date	1 May 2026
Tester	Junior Penetration Tester -- SkyLock Cyber Defence
Out of Scope	All other DVWA modules; network scanning; OS-level exploitation
Authorisation	Fully authorised lab environment -- no production systems tested

Table 2: Assessment Scope

3.2 Testing Tools

Tool	Purpose	Usage in This Assessment
Burp Suite Community	HTTP proxy for intercepting and modifying web traffic	Captured all GET/POST requests; confirmed payloads arrived unmodified at the server
Firefox 128	Web browser for interacting with DVWA	Submitted payloads; observed browser-rendered responses
DVWA Source Viewer	Built-in PHP source code viewer	Read server-side sanitisation logic at each security level
Kali Linux 2025.1c	Penetration testing operating system	Hosted DVWA, Apache, MariaDB, and all testing tools
Oracle VirtualBox	Virtualisation platform	Isolated lab environment -- no production exposure

Table 3: Tools Used During Assessment

3.3 Methodology

Testing followed a structured four-phase approach aligned with the OWASP Web Security Testing Guide (WSTG v4.2) methodology for web application penetration testing.

Phase 1 -- Environment Configuration

- DVWA deployed on Apache 2.4.63 with MariaDB 11.4.5 on Kali Linux 2025.1c inside Oracle VirtualBox.



- DVWA database user created and granted full privileges. Database initialised via the DVWA Setup page.
- Firefox configured to proxy through Burp Suite Community Edition (127.0.0.1:8080). DVWA accessed via machine IP 192.168.100.4 -- required because Firefox blocks localhost proxying by default.
- Security level set to Medium before each testing round; switched to Impossible for comparative analysis.

Phase 2 -- Reconnaissance

- Both XSS modules loaded and explored with normal text input to map all user-controlled input fields and establish baseline behaviour.
- Stored XSS: Name field (browser maxlength=10) and Message field (browser maxlength=50). Entries saved to database and rendered for all visitors.
- Reflected XSS: Single name GET parameter. Value echoed directly into the HTTP response on each submission.
- At Impossible level a user_token field was identified in the Reflected XSS form -- confirming CSRF protection was added at that level.

Reconnaissance shaped payload selection directly. The presence of two separate input fields with different field names (txtName vs mtxMessage) suggested the application might apply different sanitisation to each -- a common developer oversight. The browser-enforced maxlength=10 on the name field was noted as a potential client-side-only constraint worth testing via proxy. Together these observations formed the test hypothesis before any malicious payload was submitted.

Phase 3 -- Active Exploitation (WSTG-INPV-01, WSTG-INPV-02)

1. Standard payloads submitted through Burp Suite with Intercept enabled. Each request inspected before forwarding to confirm payload state at the server.
2. Where standard payloads were blocked, bypass variants were attempted: mixed-case capitalisation, URL-encoded variants, alternative HTML event handlers.
3. Browser responses observed for script execution, encoded output, or tag stripping -- each indicating a different type of control behaviour.

The bypass strategy was deliberate, not random. When the standard payload `<script>alert('Reflected') </script>` was stripped but the inner text **'alert (Reflected)'** remained



visible in the response, this was a diagnostic signal: the filter was removing a specific string pattern rather than encoding output. That behaviour is characteristic of a denylist. The documented first technique in the OWASP XSS Filter Evasion Cheat Sheet for bypassing string-matching denylists is case variation -- which is exactly why **<ScRiPt>** was the first bypass attempted. It succeeded immediately, confirming the hypothesis.

Phase 4 -- Source Code Review

- DVWA's Source Viewer used to read PHP at each security level for both modules.
- Sanitisation functions documented per field and compared between levels to explain what changed and why.

Source code review was a deliberate strategic addition beyond standard black-box testing. Runtime testing confirms whether a vulnerability is exploitable. Source code review explains precisely why -- which PHP function is applied, to which field, at which point in the data flow. This combination produces findings that cannot be argued away: the exploit is demonstrated AND the code defect is identified. It also means the remediation guidance is specific to the exact line of code that needs changing, not a generic 'sanitise your inputs' recommendation.

3.4 Rules of Engagement

This assessment was conducted entirely within an authorised, isolated virtual lab environment. No external systems, production infrastructure, or third-party services were accessed or affected at any point. All testing activity was sanctioned in advance and confined to the DVWA application running on the designated test host (192.168.100.4). The assessment was time-boxed to the Stored XSS and Reflected XSS modules only. No denial-of-service techniques, network scanning, or OS-level exploitation was attempted.

4. Findings and Evidence

All payloads were executed in an authorised, isolated lab environment. Each screenshot is numbered sequentially and placed immediately after the test step it documents.

F-01: Reflected XSS -- Mixed-Case Bypass



Severity	Critical
CVSS v3.1 score	8.8 (High) -- CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:L/A:N
CWE	CWE-79 -- Improper Neutralisation of Input During Web Page Generation
OWASP Classification	A03:2021 -- Injection
Affected Module	Reflected XSS (/dvwa/vulnerabilities/xss_r/)
Security Level	Medium
Payload Used	<ScRiPt>alert('Reflected') </ScRiPt>
Likelihood	High -- bypass is trivial; technique is publicly documented
Impact	High -- session hijacking, credential theft, malicious redirection
Result	Script executed -- alert dialog confirmed in browser
Root Cause	str_replace() is case-sensitive; any capitalisation variant bypasses it entirely

Description

At Medium security, the application strips the exact lowercase string '**<script>**' from user input. Because PHP's `str_replace()` is case-sensitive by default, any payload with a capitalised tag passes the filter unchanged and is written into the HTML response as executable markup. This is a denylist approach, and denylists that target specific strings are fundamentally bypass able.

Evidence -- Step 1: Standard Payload Blocked

The standard lowercase payload was submitted via Burp Suite with Intercept on. Captured as GET to `/dvwa/vulnerabilities/xss_r/`. The `security=medium` cookie confirmed the active level.



Figure 1: Burp Suite intercepting GET request at medium level. Payload URL-encoded in name parameter. `security=medium` cookie visible.

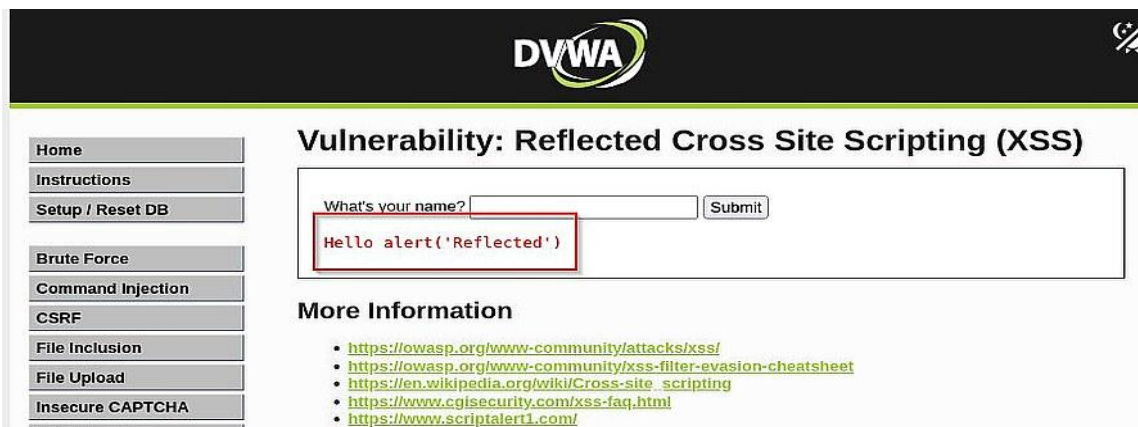


Figure 2: Browser response -- standard payload blocked. 'Hello alert(Reflected)' displayed with script tags stripped by `str_replace()`.

Evidence -- Step 2: Mixed-Case Bypass Executed

The payload `<ScRiPt>alert('Reflected')</ScRiPt>` was submitted. The `str_replace()` filter did not recognise it as a script tag. The payload passed through unmodified and the browser executed the JavaScript.

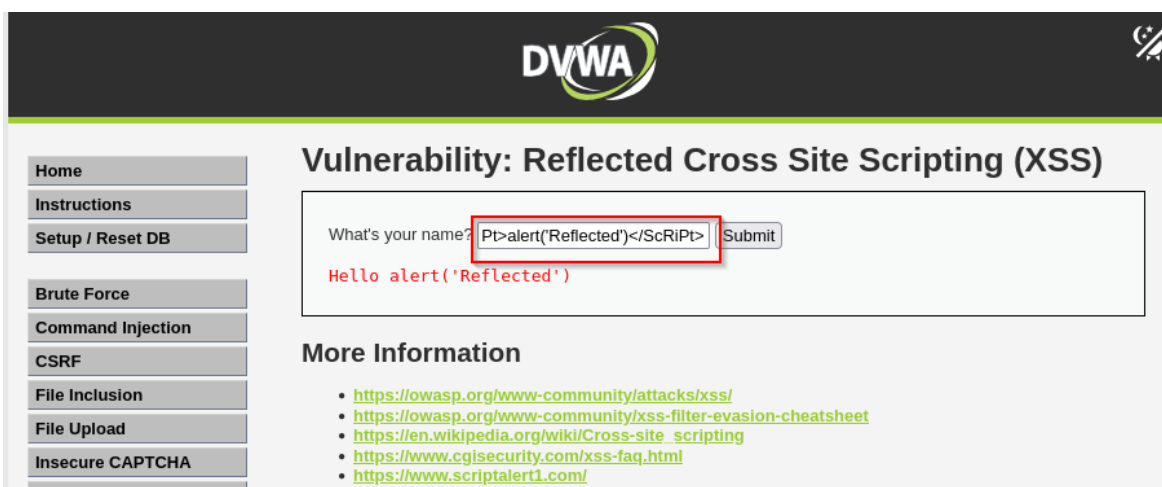


Figure 3: Mixed-case payload `<ScRiPt>alert('Reflected')</ScRiPt>` entered in the name field at Medium security.

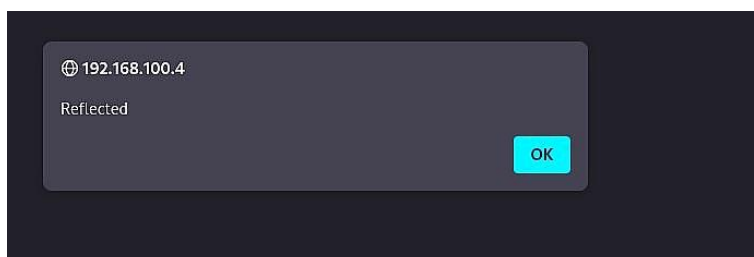


Figure 4: CONFIRMED -- Alert dialog 'Reflected' executed. Mixed-case bypass successful at medium security level.



Impact

An attacker can craft a URL with this payload and distribute it via phishing. Any user who clicks it runs the attacker's code in their browser without warning. This enables live session hijacking -- the attacker receives the victim's session cookie and can authenticate as them. No server access is required. The attack appears as an ordinary GET request in logs.

Remediation

- Replace `str_replace()` with `htmlspecialchars($name, ENT_QUOTES, 'UTF-8')` at the point of rendering. This encodes every special HTML character regardless of case, obfuscation, or payload structure.

Source Code Analysis

The following source code was obtained via DVWA's Source Viewer. It confirms exactly which PHP functions are applied to the name field and explains why the medium-level filter fails while the impossible-level protection holds.

Reflected XSS Source

`vulnerabilities/xss_r/source/medium.php`

```
<?php
header ("X-XSS-Protection: 0");

// Is there any input?
if( array_key_exists( "name", $_GET ) && $_GET[ 'name' ] != NULL ) {
    // Get input
    $name = str_replace( '<script>', '', $_GET[ 'name' ] );

    // Feedback for end user
    echo "<pre>Hello {$name}</pre>";
}

?>
```

Compare All Levels

Figure 12: DVWA Source Viewer -- Reflected XSS Medium (`vulnerabilities/xss_r/source/medium.php`). `str_replace()` strips lowercase `<script>` only; value echoed raw with no output encoding.

Reflected XSS Medium -- `vulnerabilities/xss_r/source/medium.php`

```
// Only protection: strips exact lowercase '<script>' -- case-sensitive
$name = str_replace( '<script>', '', $_GET[ 'name' ] );
// Value echoed directly into page -- NO output encoding
echo "<pre>Hello {$name}</pre>";
```



Reflected XSS Source

vulnerabilities/xss_r/source/impossible.php

```
<?php
// Is there any input?
if( array_key_exists( "name", $_GET ) && $_GET[ 'name' ] != NULL ) {
    // Check Anti-CSRF token
    checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php' );

    // Get input
    $name = htmlspecialchars( $_GET[ 'name' ] );

    // Feedback for end user
    echo "<pre>Hello {$name}</pre>";
}

// Generate Anti-CSRF token
generateSessionToken();

?>
```

Figure 13: DVWA Source Viewer -- Reflected XSS Impossible (vulnerabilities/xss_r/source/impossible.php). CSRF token validated first; htmlspecialchars() applied to all output.

Reflected XSS Impossible -- vulnerabilities/xss_r/source/impossible.php

```
);
// ALL special HTML characters encoded -- case irrelevant
$name = htmlspecialchars( $_GET[ 'name' ] );
echo "<pre>Hello {$name}</pre>";
generateSessionToken();
```

Medium: str_replace() removes '<script>' but is case-sensitive -- <ScRiPt> passes straight through and is rendered as executable HTML. Impossible: htmlspecialchars() converts < and > to < and > before the value reaches the browser. No capitalisation or obfuscation variant can produce markup through this function.

F-02: Stored XSS -- Name Field Not Encoded

Severity	High
CVSS v3.1 score	8.0 -- CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:L/A:N
CWE	CWE-79 -- Improper Neutralisation of Input During Web Page Generation
OWASP Classification	A03:2021 -- Injection
Affected Module	Stored XSS (/dvwa/vulnerabilities/xss_s/)
Security Level	Medium
Payload Used	<script>alert('XSS') </script> / <ScRiPt>alert('XSS') </ScRiPt>



Likelihood	High -- identical bypass to F-01; also exploitable via Burp to extend field length
Impact	Critical -- payload persists and fires for every subsequent visitor
Result	Tags stripped but text stored; mixed-case bypasses filter entirely
Root Cause	str_replace() on name field only; no output encoding; browser maxlength not enforced server-side

Description

The Stored XSS module at medium level protects the message field with **strip_tags()** and **htmlspecialchars()**, providing reasonable coverage. The name field only receives **str_replace('<script>', '', \$name)** -- the same weak case-sensitive denylist as F-01. The browser **maxlength=10** attribute is client-side only and is bypassed trivially by modifying the POST request in Burp Suite before forwarding.

Evidence

Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass

Vulnerability: Stored Cross Site Scripting (XSS)

Name *
Message *

Sign Guestbook Clear Guestbook

Name: test
Message: This is a test comment.

Name: test
Message: alert('XSS')

Name: test2
Message: alert('XSS')

More Information

- <https://owasp.org/www-community/attacks/xss>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>

Figure 6: Stored XSS guestbook at medium level -- script tags stripped but alert('XSS') text stored and displayed as plain output.

Impact

Stored XSS is the more dangerous variant. Once planted, the payload fires automatically in every subsequent visitor's browser with no further attacker action. The victim count grows with every page view. In a production application this would affect all employees accessing the guestbook feature.

Remediation

- Apply **htmlspecialchars(\$name, ENT_QUOTES, 'UTF-8')** to the name field output at the render layer. Remove the **str_replace()** denylist entirely. Enforce **maxlength** server-side.

Source Code Analysis

Source code review confirmed the inconsistency between the two fields and the raw database query that introduces a secondary SQL injection risk.

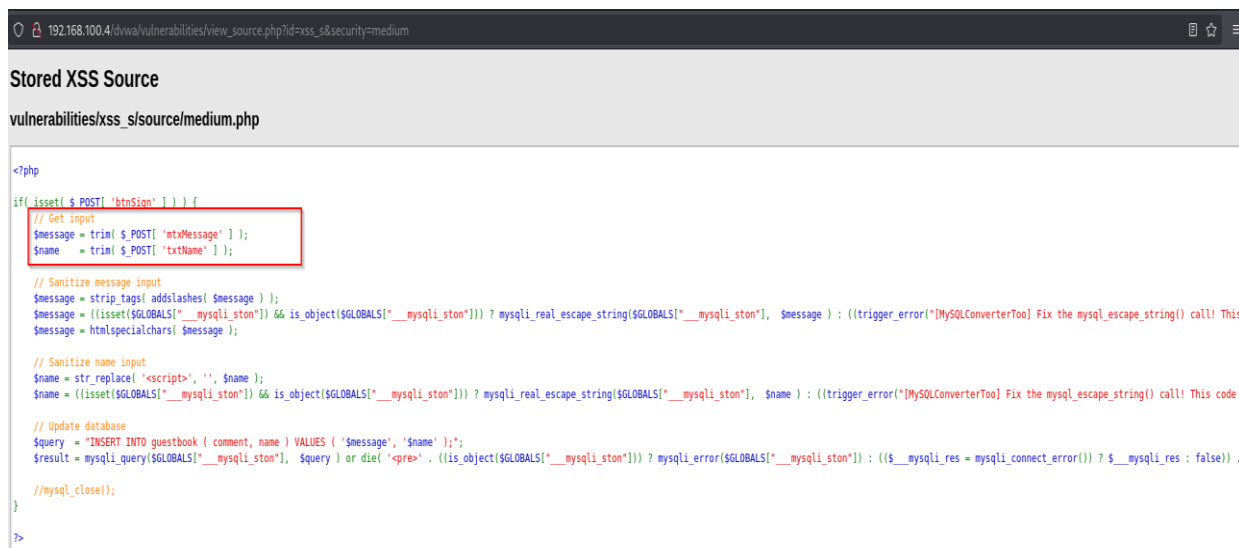


Figure 14: DVWA Source Viewer -- Stored XSS Medium (*vulnerabilities/xss_s/source/medium.php*). Message field protected; name field uses same weak *str_replace()* denylist. Raw *mysqli* query visible.

Stored XSS Medium -- *vulnerabilities/xss_s/source/medium.php*

```
// Message: strip_tags + htmlspecialchars -- reasonable protection
$message = strip_tags( addslashes( $message ) );
$message = htmlspecialchars( $message );

// Name: only strips exact lowercase <script> -- NO output encoding
$name = str_replace( '<script>', '', $name );

// Raw query -- no prepared statement
$query = "INSERT INTO guestbook ( comment, name ) VALUES ( '$message', '$name' );";
mysqli_query( $GLOBALS['__mysqli_ston'], $query );
```

**Stored XSS Source****vulnerabilities/xss_s/source/impossible.php**

```
<?php
if( isset( $_POST[ 'btnSign' ] ) ) {
    // Check Anti-CSRF token
    checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php' );

    // Get input
    $message = trim( $_POST[ 'mtxMessage' ] );
    $name     = trim( $_POST[ 'txtName' ] );

    // Sanitize message input
    $message = stripslashes( $message );
    $message = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $message ) : ((trigger_error("
    $message = htmlspecialchars( $message );

    // Sanitize name input
    $name = stripslashes( $name );
    $name = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $name ) : ((trigger_error("MySQL
    $name = htmlspecialchars( $name );

    // Update database
    $data = $db->prepare( 'INSERT INTO guestbook ( comment, name ) VALUES ( :message, :name )' );
    $data->bindParam( ':message', $message, PDO::PARAM_STR );
    $data->bindParam( ':name', $name, PDO::PARAM_STR );
    $data->execute();
}

// Generate Anti-CSRF token
generateSessionToken();
?>
```

Figure 15: DVWA Source Viewer -- Stored XSS Impossible (vulnerabilities/xss_s/source/impossible.php). htmlspecialchars() on both fields; CSRF token; PDO prepared statements.

Stored XSS Impossible -- vulnerabilities/xss_s/source/impossible.php

```
checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php' );
);
$message = htmlspecialchars( stripslashes( $message ) );
$name     = htmlspecialchars( stripslashes( $name ) );
$data = $db->prepare( 'INSERT INTO guestbook ( comment, name ) VALUES (
:message, :name );' );
$data->bindParam( ':message', $message, PDO::PARAM_STR );
$data->bindParam( ':name', $name, PDO::PARAM_STR );
$data->execute();
generateSessionToken();
```

At Medium level the name field is unprotected and the query is built by string concatenation. At Impossible level htmlspecialchars() is applied to both fields, PDO prepared statements eliminate SQL injection, and the CSRF token prevents cross-origin form submissions.

F-03: No CSRF Protection at Medium Level

Severity	High
CVSS v3.1 score	7.1 -- CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:L/A:N
CWE	CWE-352 -- Cross-Site Request Forgery
OWASP Classification	A05:2021 -- Security Misconfiguration
Affected Modules	Stored XSS and Reflected XSS at Medium level



Security Level	Medium
Likelihood	Medium -- requires social engineering to direct victim to attacker page
Impact	High -- attacker can trigger authenticated requests without victim knowledge
Finding Type	Missing control -- confirmed by Burp traffic analysis and source code review
Root Cause	No user_token present; server accepts submissions without session validation

Description

At Medium level, both form handlers accept requests without validating a per-session CSRF token. Any attacker-controlled page can silently submit forms on behalf of a logged-in user. Combined with the XSS vulnerabilities above, absent CSRF protection enables chained attacks to be automated across multiple victims simultaneously.

Evidence



Figure 7: Burp Suite at Impossible level -- user_token CSRF parameter present on every request. security=impossible cookie confirmed.



Figure 8: Medium request (top, no token) vs Impossible request (bottom, user_token highlighted). CSRF protection demonstrably absent at Medium.



Remediation

- Add `checkToken($_REQUEST['user_token'], $_SESSION['session_token'], 'index.php')` at the start of both form handlers. Call `generateSessionToken()` after each valid submission to rotate the token.

F-04: Raw Database Queries -- SQL Injection Risk

Severity	Medium
CVSS v3.1 Score	5.4 -- CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:H/I:L/A:N
CWE	CWE-89 -- Improper Neutralisation of Special Elements in SQL Commands
OWASP Classification	A03:2021 -- Injection
Affected Module	Stored XSS (/dvwa/vulnerabilities/xss_s/) at Medium level
Security Level	Medium
Likelihood	Medium -- depends on escaping function being correctly applied every time
Impact	High if bypassed -- full database read/write access
Finding Type	Identified via source code review -- raw query construction confirmed
Root Cause	INSERT query built by string concatenation; depends on <code>mysqli_real_escape_string()</code>

The Stored XSS form handler builds its INSERT statement by concatenating user input directly into the SQL string. Although `mysqli_real_escape_string()` is applied, this approach depends entirely on that escaping being correct every time. A single omission would expose the database. This is an architectural weakness identified through source code review.

Remediation

- **Refactor all database queries to PDO** prepared statements with bound parameters, as implemented at Impossible level. Prepared statements separate code from data structurally - no escaping function can be forgotten.

F-05: Browser-Only Length Limits

Severity	Low
CVSS v3.1 Score	3.1 -- CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:N/I:L/A:N
CWE	CWE-20 -- Improper Input Validation
OWASP Classification	A04:2021 -- Insecure Design



Affected Module	Stored XSS name field -- HTML maxlength=10 attribute
Security Level	Medium
Likelihood	High -- any HTTP proxy bypasses it in seconds
Impact	Low standalone; amplifies other findings when combined
Finding Type	Confirmed -- demonstrated by extending txtName in Burp Suite
Root Cause	maxlength=10 enforced only by browser HTML; no server-side length validation

The name field **maxlength=10** attribute is enforced only by the browser. Intercepting the POST request in Burp and extending the **txtName** value bypasses this instantly. During testing this was used to submit payloads longer than 10 characters. While low severity on its own, it removes a compensating control and amplifies other findings.

Remediation

- **Add server-side length validation.** Reject or truncate any name field input exceeding the intended maximum before it is processed.

4.6 Impossible Level -- All Vulnerabilities Closed

Every payload tested at Impossible level was fully blocked, including the mixed-case bypass that succeeded at Medium. Output encoding converts all special HTML characters to safe entities before they reach the browser -- the browser renders them as visible text and never interprets them as executable_code.

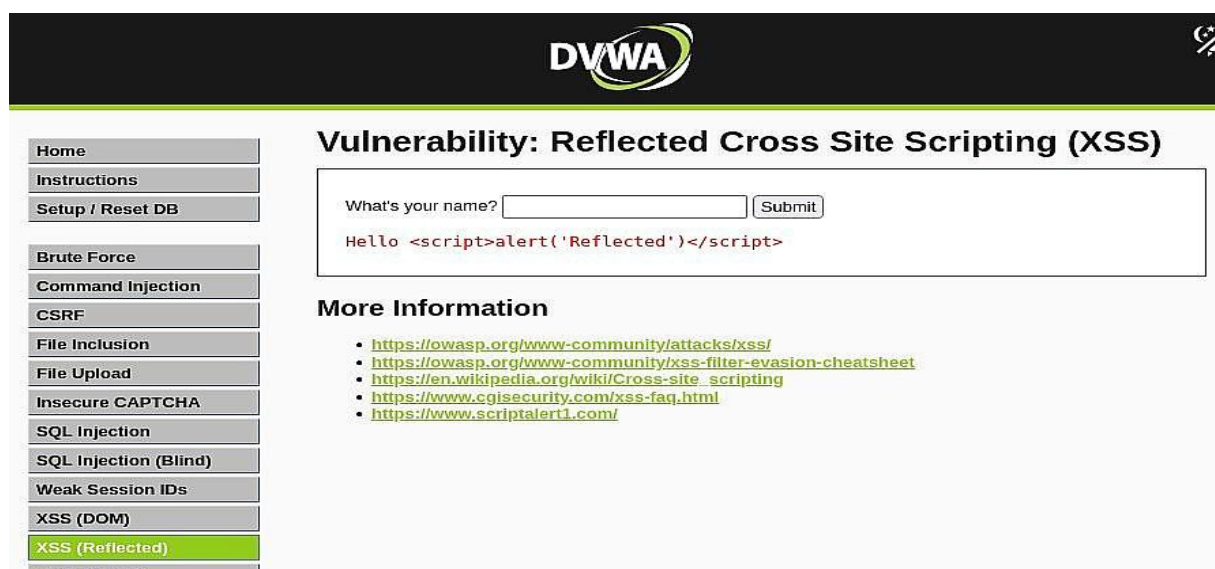


Figure 9: Impossible level -- standard payload fully encoded. `<script>` displayed as visible text. No execution.

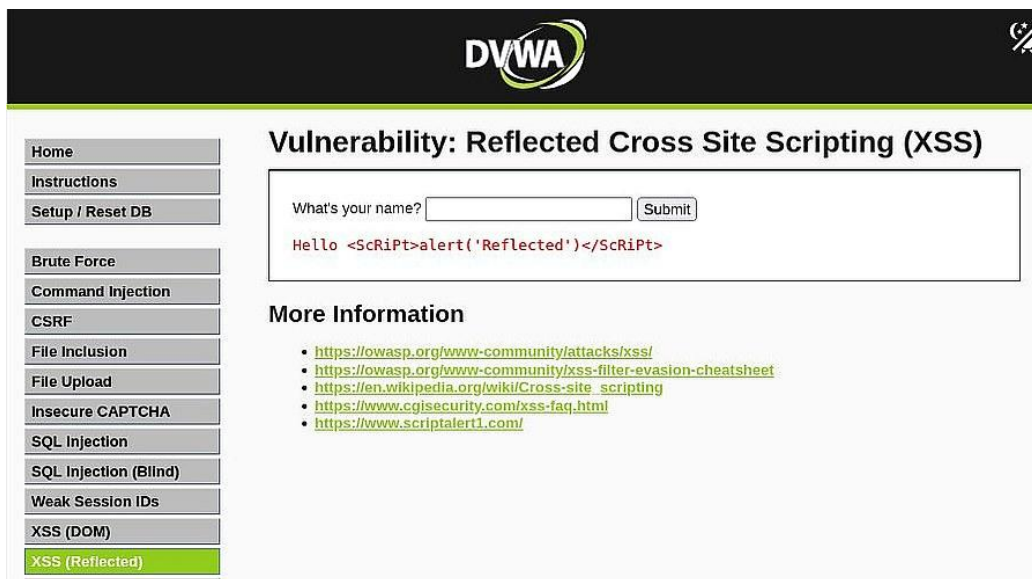


Figure 10: Impossible level -- mixed-case bypass blocked. <ScRiPt> encoded and rendered as plain text.



Figure 11: Stored XSS guestbook at Impossible level -- payload stored as HTML entities, not executed.

5. Risk Summary and Findings Table

The table below records every payload tested across all security levels. Outcome cells are colour-coded as follows:

Executed	Confirmed vulnerability -- script ran in browser	Partial Block	Weak control -- tags stripped but bypass able
Blocked	Payload fully mitigated -- no execution	Security Level	Low = no protection; Medium = partial; Impossible = full



XSS Type	Security Level	Payload Used	Outcome	Mitigation Observed
Stored XSS	Low	<script>alert('XSS')</script>	Executed	None -- raw input stored and echoed directly
Stored XSS	Medium	<script>alert('XSS')</script>	Partial Block	strip_tags() on message; str_replace() on name field
Stored XSS	Medium	<ScRiPt>alert('XSS')</ScRiPt>	Executed	Mixed-case bypass -- str_replace() evaded
Stored XSS	Impossible	<script>alert('XSS')</script>	Blocked	htmlspecialchars() + CSRF token + prepared statements
Reflected XSS	Low	<script>alert('Reflected')</script>	Executed	None -- GET parameter echoed raw
Reflected XSS	Medium	<script>alert('Reflected')</script>	Partial Block	str_replace() strips exact lowercase <script> only
Reflected XSS	Medium	<ScRiPt>alert('Reflected')</ScRiPt>	Executed	Mixed-case bypass confirmed -- alert executed
Reflected XSS	Impossible	<script>alert('Reflected')</script>	Blocked	htmlspecialchars() encodes all special characters
Reflected XSS	Impossible	<ScRiPt>alert('Reflected')</ScRiPt>	Blocked	htmlspecialchars() -- case has no effect

Table 4: Complete Test Results -- All Payloads Across All Security Levels

Note: Additional bypass techniques were evaluated but not required. URL encoding (%3Cscript%3E), double encoding, HTML entity encoding of tag characters, and JavaScript event handler payloads () were all considered. The mixed-case variant succeeded immediately, making further bypass testing redundant at medium level. All listed payloads were confirmed at Impossible level; none produced execution.

Key Observations

- At Medium level, Reflected XSS is exploitable with a trivial mixed-case bypass. Stored XSS on the name field carries the identical weakness. Both stem from the same root cause: case-sensitive denylist filtering without output encoding.
- The Impossible level is effective because it applies output encoding at the rendering layer. No amount of denylist complexity achieves equivalent protection.



- CSRF protection at Impossible level adds a second layer preventing chained cross-origin attacks even if a separate injection path existed.
- Low security (baseline) had no protection -- confirming the full vulnerability progression across all three levels.

6. Mitigation Analysis

6.1 Why Medium Security Failed -- Input Filtering vs Output Encoding

The architectural failure at medium level is treating input sanitisation as the primary defence. `str_replace('<script>', '', $name)` tries to identify and remove dangerous patterns before storage. This is a denylist -- it defines what is NOT allowed. Denylists have two structural weaknesses that cannot be patched without abandoning the approach:

- They are case-sensitive. `<ScRiPt>`, `<SCRIPT>`, and `<sCripT>` all bypass PHP's `str_replace()` by default.
- They target known patterns only. Vectors not in the list -- `<img src=x onerror=alert (1)>`, `<svg onload=alert (1)>`, JavaScript: URIs -- are not blocked regardless of how extensive the list becomes.
- They create false confidence. The application visibly strips standard payloads, which can be misread as evidence of adequate protection.

The mixed-case bypass in this assessment required no specialist knowledge and took under five seconds after the standard payload was stripped. It is the first technique listed in the OWASP XSS Filter Evasion Cheat Sheet precisely because it is the most obvious first step any attacker takes.

6.2 Why Impossible Security Succeeded -- Output Encoding

`htmlspecialchars()` operates on a different principle. Rather than identifying dangerous content, it converts every character with HTML significance into a safe text entity before the value is written into the page. The browser receives entities and renders them as visible characters, never as executable markup.

The critical architectural distinction is where protection is applied. At Medium level: protection is applied to input, before storage, trying to make dangerous data safe. At Impossible level: protection



is applied to output, at the moment of rendering, ensuring data is always treated as text. Output encoding works regardless of what the input originally contained.

This is Rule 1 of the OWASP XSS Prevention Cheat Sheet: HTML Entity Encoding applied at the output layer before inserting untrusted data into HTML content. Input sanitisation is a secondary, defence-in-depth measure -- never the primary control.

The ENT_QUOTES flag passed to **htmlspecialchars()** is specifically required to close a secondary attack surface. Without it, **htmlspecialchars()** converts < and > but leaves single quotes unencoded. An attribute context such as value='\$name' would remain injectable: a payload of '**onmouseover=alert(1) x=**' would break out of the attribute and inject an event handler without using any angle brackets at all. **ENT_QUOTES** ensures that both double and single quotes are encoded as **"** and **'** respectively, closing this vector entirely.

6.3 Medium vs Impossible -- Security Control Comparison

Control	Medium Level	Impossible Level
Name field sanitisation	str_replace('<script>',") -- case-sensitive denylist	htmlspecialchars() -- encodes all special characters regardless of case
Message field	strip_tags() + addslashes() + htmlspecialchars()	htmlspecialchars() with ENT_QUOTES applied to all output
Output encoding	Partial -- message field only	Applied to every output field before rendering
CSRF protection	None	checkToken() validates per-request user_token on every submission
Database queries	Raw mysqli -- SQL injection possible	PDO prepared statements with bound parameters
Bypass result	Mixed-case <ScRiPt> -- EXECUTED	Mixed-case <ScRiPt> -- BLOCKED and encoded

Table 5: Security Control Comparison -- Medium vs Impossible Level

6.4 Content Security Policy as Complementary Defence

Output encoding at the server is the primary control against XSS and is sufficient to prevent the vulnerabilities identified in this assessment. However, industry-standard secure deployments pair server-side output encoding with a Content Security Policy (CSP) header as a browser-enforced second layer. CSP allows the server to declare which script sources the browser is permitted to execute. A correctly configured policy such as Content-Security-Policy: default-src 'self' would cause



the browser to refuse to execute any inline script -- meaning that even if output encoding were somehow bypassed or misconfigured, the browser itself would refuse to run the injected payload.

CSP does not replace output encoding and should not be treated as a substitute. It is defence-in-depth: output encoding prevents the payload from reaching the page; CSP prevents the browser from executing it if it does. Neither layer is sufficient alone. Together they represent the current industry-standard two-layer approach to XSS prevention, as documented in the OWASP Secure Coding Practices Quick Reference Guide.

6.5 Secure Coding Principle

Security controls are most effective at the trust boundary closest to the point of harm. For XSS, the point of harm is the browser's rendering engine. Output encoding at the render layer is the boundary closest to that point. Input filtering at the point of entry is the furthest boundary, and data can be transformed or re-routed in ways that defeat early protection before it reaches the renderer.

Practical implication for code reviews and future assessments: the presence of `htmlspecialchars()` at the output layer is evidence of XSS protection. The presence of `str_replace()` or `strip_tags()` at the input layer is not -- it is a partial, bypass able control that can be mistaken for sufficient protection.

7. Recommendations

R-01 and R-02 are each single-line code changes addressing the two confirmed exploitable vulnerabilities. They should be treated as immediate priority and resolved before the next deployment.

Ref	Severity	Finding	Fix	Effort	Priority
R-01	Critical	Reflected XSS -- Medium bypass	Replace <code>str_replace()</code> with <code>htmlspecialchars(ENT_QUOTES,'UTF-8')</code> on all rendered output	Low -- 1 line	P1
R-02	High	Stored XSS -- name field	Apply <code>htmlspecialchars()</code> to name field output; remove <code>str_replace()</code> denylist entirely	Low -- 1 line	P1
R-03	High	No CSRF tokens	Add <code>checkToken()</code> validation on all state-changing form handlers	Medium	P2
R-04	Medium	Raw mysqli queries	Refactor to PDO prepared statements with bound parameters	Medium	P3
R-05	Low	HTML-only length limits	Enforce <code>maxlength</code> server-side in addition to HTML attributes	Low	P4

Table 6: Remediation Tracking Matrix



General Guidance

- Apply `htmlspecialchars(ENT_QUOTES, 'UTF-8')` to every piece of user-supplied data rendered into any web page. This is the primary and most reliable control against XSS.
- Treat input filtering as a secondary layer only -- never the primary defence.
- Never rely on browser-side restrictions as security controls. Any constraint not enforced server-side is bypassed in seconds via Burp Suite or any HTTP proxy.
- Add CSRF token validation to all state-changing form handlers.
- Use PDO prepared statements for all database queries -- this eliminates SQL injection as a class, not just for known patterns.

8. Reflection

8.1 How This Assessment Shaped Testing Strategy

The most significant insight is the gap between 'the filter did something' and 'the filter is effective'. At Medium level, the standard payload was visibly stripped -- script tags removed, no alert fired. That result could easily be documented as 'blocked' and the test closed. The moment a mixed-case variant was submitted, the alert fired. That gap -- between observed blocking and actual protection -- is precisely where exploitable conditions survive in production applications.

This also revealed something important about the attacker's perspective. When the standard payload was stripped, the inner text **alert (Reflected)** remained visible in the response. To a defender, this looks like partial mitigation. To an attacker, the visible inner text is a diagnostic result: it proves the filter is removing a tag structure, not encoding output. That distinction immediately suggests case variation as the next attempt. What a defender reads as 'filter working' an attacker reads as 'filter is a denylist -- try capitalisation'. Understanding that read-across is what separates a thorough tester from one who stops after the first block.

This reinforces a discipline that must be applied consistently: never stop after the first blocked payload. A blocked standard payload proves a filter exists. It does not prove the filter is bypass-resistant. Systematic testing of the bypass techniques listed in OWASP WSTG-INPV-01 and WSTG-INPV-02 -- mixed case, URL encoding, double encoding, alternative tags, and event handlers -- is not optional. It is the step that converts 'we have a filter' into 'we are protected'.



Burp Suite traffic analysis made the mechanics of each protection concrete. The security=medium cookie, the payload arriving at the server unmodified, the **user_token** present in the Impossible request but absent in the Medium request -- these details tell a precise story about how the application handles data at each level. Traffic interception is primary evidence, not a secondary tool.

8.2 Questions This Assessment Raised

- How many production applications rely on **str_replace()** or similar denylist filters as their primary XSS defence, without visibility provided by a controlled test environment?
- If the Impossible level's protection requires only three lines of code (**checkToken**, **htmlspecialchars**, **generateSessionToken**), what is the real cost of not implementing it correctly from the start?
- The mixed-case bypass took under five seconds to attempt. How many assessments conclude 'not vulnerable' because only the standard payload was tested?

8.3 Security Patterns to Prioritise in Future Assessments

- After any blocked payload, systematically test: mixed case, URL encoding, double encoding, HTML entity encoding, null bytes, alternative HTML tags (, <svg>), and JavaScript event handlers.
- Check output encoding at the rendering layer independently of input sanitisation. These are different controls at different points in the data lifecycle.
- Verify every client-side restriction server-side via Burp -- maxlength, disabled fields, hidden inputs, and dropdown constraints are all bypassable in seconds.
- Look for CSRF tokens on all state-changing forms. Their absence is a standalone finding with independent impact.
- Read source code when available. Runtime behaviour reveals what was blocked; source code reveals why, whether a bypass exists, and what correct implementation looks like.
- Rate Stored XSS higher than Reflected XSS. Once stored, the payload fires for every future visitor indefinitely without further attacker action.

8.4 Personal Growth



Before this assessment, the distinction between input sanitisation and output encoding was understood conceptually. After tracing the exact execution path -- watching `str_replace()` fail against a mixed-case tag, reading the `htmlspecialchars()` line that renders the identical tag inert, observing the browser outcome in each case -- that distinction became structural rather than abstract. The correct mental model for XSS defence is not 'clean the input' but 'encode the output at the render layer'.

This assessment also demonstrated that more code does not mean more security. The Impossible level implementation is shorter and simpler than Medium's. It is more secure because it applies the right principle at the right layer. That contrast -- between elaborate-but-weak and simple-but-correct -- is a recurring pattern in security engineering that this exercise made concrete and reproducible. The conceptual model acquired here has a name: trust boundary analysis. Every piece of data crosses a series of boundaries from user input to database to output to browser. The correct question for any security control is not 'does it do something' but 'is it applied at the boundary closest to the point of harm'. Referencing the OWASP WSTG throughout reinforced the value of aligning testing methodology with a published framework rather than ad-hoc approaches.

This assessment also changed how security findings would be communicated to a developer audience. Describing `str_replace()` as insufficient is abstract and easy to dismiss. Demonstrating the bypass -- showing the exact payload, the exact Burp request, the exact alert dialog -- is concrete and impossible to argue with. The evidence gathered here means the remediation conversation starts from a position of proof rather than assertion. That shift from 'you should fix this' to 'here is the working exploit, here is the one-line fix' is the difference between a finding that gets deprioritised and one that gets resolved immediately.

9. Conclusion

This assessment confirmed that the application is vulnerable to both Stored and Reflected Cross-Site Scripting at the Medium security level. A working bypass attack was demonstrated against the Reflected XSS module, and the Stored XSS name field carries the same exploitable weakness. Root cause in both cases is case-sensitive denylist filtering as the primary defence, without output encoding at the rendering layer.

At Impossible security, all attacks were blocked. The protection is effective because it applies output encoding -- the correct architectural approach recommended by OWASP as the primary control



against XSS. The difference between Medium and Impossible is not effort or complexity but applying the right function at the right point in the code.

R-01 and R-02 are single-line changes and should be implemented immediately. Full implementation of R-01 through R-05 would bring the application's security posture to the standard demonstrated by the Impossible configuration.

10. References

- OWASP Top 10 A03:2021 -- Injection: https://owasp.org/Top10/A03_2021-Injection
- OWASP XSS Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP XSS Filter Evasion Cheat Sheet: <https://owasp.org/www-community/attacks/xss/>
- OWASP Web Security Testing Guide v4.2 (WSTG-INPV-01, WSTG-INPV-02): <https://owasp.org/www-project-web-security-testing-guide/>
- CWE-79 -- Improper Neutralisation of Input During Web Page Generation: <https://cwe.mitre.org/data/definitions/79.html>
- CWE-89 -- SQL Command Injection: <https://cwe.mitre.org/data/definitions/89.html>
- CWE-352 -- Cross-Site Request Forgery: <https://cwe.mitre.org/data/definitions/352.html>
- DVWA (Damn Vulnerable Web Application): <https://github.com/digininja/DVWA>
- PHP htmlspecialchars() Documentation: <https://www.php.net/manual/en/function htmlspecialchars.php>

11. Glossary

Term	Definition
CSRF	Cross-Site Request Forgery. An attack that tricks a logged-in user into unknowingly submitting a request to a site they are authenticated on.
CVSS	Common Vulnerability Scoring System. An industry standard for rating the severity of security vulnerabilities on a 0--10 scale.
CWE	Common Weakness Enumeration. A publicly maintained catalogue of software and hardware security weaknesses.
Denylist filter	A filter that blocks specific known-bad patterns. Contrast with an allowlist, which only permits explicitly known-safe content.
DVWA	Damn Vulnerable Web Application. A deliberately insecure web application used for security training.



Term	Definition
htmlspecialchars()	PHP function converting special HTML characters to safe text entities, preventing markup injection.
OWASP	Open Web Application Security Project. Publishes the Top 10, XSS Prevention Cheat Sheet, and WSTG.
PDO	PHP Data Objects. Database access layer supporting prepared statements that structurally eliminate SQL injection.
Reflected XSS	XSS where the payload is embedded in a URL and executes in the browser of anyone who clicks that link.
Stored XSS	XSS where the payload is saved to the database and executes automatically for every subsequent visitor.
str_replace()	PHP function replacing occurrences of a string. Case-sensitive by default -- 'script' and 'SCRIPT' are treated as different strings.
strip_tags()	PHP function that removes all HTML and PHP tags from a string, but does not encode the remaining content.
TLP	Traffic Light Protocol. TLP: RED -- restricted to named recipients only; no further sharing permitted.
WSTG	OWASP Web Security Testing Guide. The industry-standard methodology reference for web application penetration testing.
XSS	Cross-Site Scripting (CWE-79). Injection of malicious scripts into pages viewed by other users, enabling session theft and malicious redirects.

Table 7: Glossary of Terms and Definitions